Counting circuit Package Desciption. Version 0.2

March 14, 2026

1 Introduction

Counting circuit package (countingcircuit.py, depends on gencthy.py) is intended for studying statistics of electron transport in semiclassical nanostructures and related properties of the scattering matrix of the nanostructure, such as transmission distribution. The package implements the full counting quantum circuit theory as described in Nazarov and Blanter, *Quantum Transport*, Ch. 2.9 and Nazarov, YV. and Bagrets, DA. *Physical Review Letters*, 88(19), 196801-1-196801-4, 2002. The nanostructure is modeled with a circuit that includes normal terminals biased at different voltages and (generally different) temperatures. The connectors are generally characterized by transmission distribution. Tunnel, diffusive, ballistic, and arbitrary connectors are implemented. The key quantity is the action of the circuit that is a function of *counting fields* in the terminals. Usually, the action itself cannot be directly associated with observable quantities. The package provides the tools using this action to compute:

- the cumulants of currents in terminals (average currents, noises, etc.) up to the 4th order cumulants
- the densities of eigenvalues of various transmission matrix squares that can be defined in the circuit
- the probabilities of gigantic fluctuations of currents in the terminals

The actual use of the package starts with creating an instance of the **CountingCircuit** class. Then the circuit is *built* by adding the terminals, nodes, and connectors. It is so that each element of the circuit should be given a name, and addressed by this name. If you do not like giving names, make your own script to do this for you.

The computation for a circuit with nodes involves solving the quantum circuit theory equations with an iterative method. This may take time and result in bad accuracy outputs, non-convergent iterations and even overflow errors. See the Section " Solving hints" for possible remedies.

2 Building a circuit

Import **CountingCircuit** from the package **countingcircuit**. The instance of the class, our circuit, is created by calling

CountingCircuit (energy =0.0, temperature=0.01)
This returns the class instance. Optional arguments set initial energy and global temperature of the circuit.

The following member functions of the instance add the elements of the setup without connecting them:

addTerminal (name, voltage=0.0, temperature=None,cf=0.0)
Adds a superconducting terminal with a given name. The optional parameters set the terminal voltage, the terminal temperature (if different from global temperature), and counting field **cf**.

addNode(name, G=None):
Adds a node of given name. The optional argument sets initial 2×2 Keldysh Green function in the node (usually not important, since it is recalculated in the process of iterative solving)

The following function adds a connector.

addConnector(name,code,contype,Hname,Tname,conductance=1.0,
U=None,V=None,transmissions=None):
This adds a connector of a given name. The possible values of **code** are 'TT' (connects two terminals), 'NT'(connects a node and a terminal), 'NN' (connects two nodes). The arguments **Hname**, **Tname** are the names of these nodes/terminals that must already be added to the circuit. The argument **contype** determines the type of the connector. The possible types are 'Tunnel', 'Diffusive', 'Ballistic', and 'Arbitrary'. If the type is 'Arbitrary', one should supply a list of transmission eigenvalues with optional argument **transmissions**. The optional argument **conductance** sets the conductance of the connector. The optional arguments **U** and **V** are of no use for the current version of the package.

3 Setting and changing the parameters

setEnergy (double en):
Sets the energy of all terminals to **en**. While used internally in result-giving functions, usually does not have to be called explicitly, unless one wants to inspect the action at a given energy.

setGlobalTemperature(double te):
Sets the global temperature of the setup to **te**. Note that if the temperature of some terminals has been previously set to the value that *differs* from global temperature, the function call does not change their temperatures.

setTerminal (name, voltage=None, temperature=None,cf=None)
Sets/changes the parameters of the named terminal. If a parameter is given the value **None** (which is by default), its value does not change.

setConductance(name,c):
This sets conductance of the named connector to the value **c**.

It is frequently convenient to concentrate on a situation where for some terminals all electron states are filled (filling factor 1) while in the rest of terminals all electron states are empty(filling factor 0). For instance, at vanishing temperature the whole energy space is separated into a set of strips, this situation occurs in each strip with different terminals being filled/empty. In this package, the situation is achieved by setting a *mask*:

putMask(list-of-terminals):
This sets the filling factors in the terminals from the list to 1, and the filling factors of all terminals not included to the list to 0. Putting a mask does not erase information about counting fields in the terminals, nor about their temperatures and voltages: the latter information is just ignored.

removeMask():
This removes the mask and sets the filling factors in the terminals to the values determined by the energy and their temperatures and voltages. Note: you cannot change energy while a mask is applied! Remove the mask first.

4 Getting the results

The following functions return the results. Note: all currents computed are to corresponding terminals.

actionPI ():
This function returns the action Per energy Interval given energy and terminal settings. If a mask is set, it computes the action per interval for the settings determined by the mask.

actionZ ():
This function returns the full action assuming Zero temperature but using the voltages of the terminals. Basically, the energy space is separated into the strips where a mask can be applied, the action per interval is multiplied by a strip width, and contributions of all strips are summed up.

actionINT():
This returns the total action integrated over the energies with taking into account temperatures of all terminals. Note: before using this function, one wants to check integration parameters that are stored in **integrationparameters**. Basically, it is a list of three elements. The first parameter gives the scale: a typical width of the energy strip where the electron transport takes place. This is contributed by voltages and temperature. The second parameter gives the center of this strip: is typically determined by the average voltage of the terminals.

The third parameter is integer and gives the number of points where the action per interval is computed. The default setting is **integrationparameters** = [1.0,0.0, 200] The parameters can be changed directly or with the helper function **setIntegrationParameters**(width=1.0,center=0.0, num = 200). The function typically solves the circuit several hundred times at different energies. So do not expect it to be fast.

4.1 Cumulants

The following functions provide the cumulants of the current distribution by computing (higher-order) derivatives of the action. In the current version, the cumulants up to fourth order are implemented.

currents(list,method='PI') :
noises(list,method='PI') :
thirdcumulants(list,method='PI') :
fourthcumulants(list,method='PI') :

Similar to the action, the functions provide the cumulants per energy interval (**method**='PI'), at zero temperature(**method**='Z'), and integrated over the energies (**method**='INT', in this case, you may want to check **integrationparameters** as described for the action). Each function returns ALL cumulants of the currents in the terminals included in the **list**. If N_l is the number of elements in the list, the function for cumulant of the order n returns $(N_l)^n$ values arranged in a n-dimensional **np.array**. For instance, **currents** returns N_l average currents, **noises** returns $(N_l)^2$ noise correlators S_{ij}, i, j labeling the terminals. The order of indices corresponds to the order of terminals in the list. Ideally, the n-dimensional array should be symmetric with respect to permutation of indices, for instance, $S_{ij} = S_{ij}$. The asymmetry of returned values indicate the accuracy of calculation. If the accuracy is no good, try to change the parameters as described in the Section "Solving tips".

4.2 Transmission distributions

The statistics of the electron transport is ultimately related to the properties of scattering matrix of the setup. In semiclassical circuits, those are eigenvalue densities of the various transmission matrices. So that, these eigenvalue densities can be accessed with this package. Generally, one can define a partial transmission matrix $\hat{t}_{AB}$ for scattering amplitudes of electrons in all channels of the set A that go to the channels of the terminals of the set B, and quantify the eigenvalue density of $\hat{t}_{AB}\hat{t}^{\dagger}_{AB}$. This is computed by the following function:

partialTransmissionDistibution(A,B,list-of-transmissions,integratedd=False)
Here, A and B are the lists of terminals in both groups (that should not have common elements). The function takes as an argument the list of transmissions T: points where the density $\rho(T)$ shall be evaluated. If **integratedd** is False,

the function returns a list of densities. If **integratedd** is True, it returns the list of *integrated* densities $\int_T^1 dT' \rho(T')$ (actually, this is a faster and more accurate calculation). If **integratedd** is 'both', it returns a list of two-element lists with both quantities, the integrated density coming first.

An important case is when the set B consist of all terminals not in A. In this case, the nanostructure can be regarded as two-terminal one and t_{AB} is a full transmission matrix of this two-terminal setup. This is computed by

fullTransmissionDistibution(A,list-of-transmissions,integratedd=False)
in a similar way.

4.3 Gigantic fluctuations

Two functions provide information about the (exponentially small) probability of gigantic fluctuations, that is, significant deviations of the currents from their average values.

gfIs (dic,method='PI',bound=5.0)
This function computes the probability of gigantic fluctuation given the values of the currents in a set of terminals. The first argument is a dictionary made of pairs **name_of_a_terminal : value_of_a_current**. The **method** determines whether the calculation is per energy interval, at zero temperature, or integrates over the energies, similar to cumulant functions. The calculation involves the optimization of the action over imaginary counting fields. It may happen that the probability is strictly zero for ranges of the current values: this is frequent at low temperatures. In this case, the optimization procedure would diverge looking for the optimum at infinitely large imaginary counting fields . To prevent this unfortunate development, the optimization is restricted to all counting fields in the interval (-bound, bound). The default values of these boundaries can be changed with the optional argument **bound**.

The function returns three objects: i. the value of the *log of the probability* ii. an array of imaginary *counting fields* at which the optimum is achieved. If any of the values in the array is close to $\pm$ **bound**, it is not the true optimum: the optimization is stopped at the boundary. This may indicate zero probability. Increase **bound** to prove or disprove this iii. an array of the *currents in ALL terminals*. This is because fixing the values of the currents in the terminals of the list **list** also conditions the values of the currents in all other terminals.

Sometimes it is faster to study the gigantic fluctuations fixing the imaginary counting fields rather than the currents. The probability is computed by the following function

gfRay(chis,ray,method='Z'):
Here, **chis** is a sequence of real numbers (suppose its lenght is N), **ray** is a vector of dimension of the number of terminals. The probability is computed in N points in the space of the counting fields given by chi[i]*ray, $0 \geq i < N$ using the **method** specified. The function returns two lists. In the first list, the log of propability in N points is stored. Each element of the second list is an

array of dimension of the number of terminals, it contains the currents in ALL terminals. There are N such elements corresponding to the points in counting field space.

4.4 Node inspection

inspectNode(name)
This returns two values characterizing the state of the node as computed from Keldysh matrix voltage of the node at given energy and counting field settings in the terminals. The first value is the *filling factor* in the node, the second value is the value of the local *counting field*. These values are complex at general settings of the counting fields. N.B.Before inspecting the nodes at certain energy, one must use setEnergy() to set the energy and solve() to find the matrix voltages at the nodes.

5 Addressing functions

These functions are not supposed to be used but may appear useful for more involved applications.

terminal(name)
Returns the object associated with the named terminal. One may inspect the parameters like voltage, temperature etc. **Do not try to change** them, use setting functions instead! One can also assign custom properties.

node(name)
The same for the named node.

connector(name)
The same for the named connector.

6 Solving tips

The heart of the class is the function **solve**() that solves iteratively the Keldysh matrix voltages in the nodes. The iterative process and reports about it are controlled by the parameters in ClassInstance.solvepars. You may change these parameters in case of problems with convergence.

- **solvepars.verbose** Integer number defining the level of reporting. 0 - no reports whatever happens. 1 - reports if the desired accuracy is not achieved after maximum number of iterations. 2 - report the results of each iteration cycle. 3 - reports the results of *each* iteration. Default is 1.
- **solvepars.maxiter** Maximum number of iterations per cycle. Increase if problems with convergence.

- **solvepars.goal** Accuracy to achieve in the iteration cycle. Decrease if no time to wait for results of better accuracy.

If the iterations result in overflow errors (they are not caught, so execution stops) for some parameters, the iterative procedure may be too aggressive. This is controlled by property adjust for each node. Use the function **changeAdjust**(newvalue) to set it to lower values in the interval (0,1). The default value is 0.5. Naturally, the calculations will take more time.

A less intuitive method to achieve the convergence is to split a node or several nodes. In this case, a node is replaced by two that are connected by a connector with a conductance larger than all other conductances of the setup. This does not affect much the results provided the conductance is sufficiently large (say, a factor of 20-50 bigger than other conductances) but may drastically improve the convergence.

Check the example file *CountingExamples.py* for examples of building and use. Happy computing!